

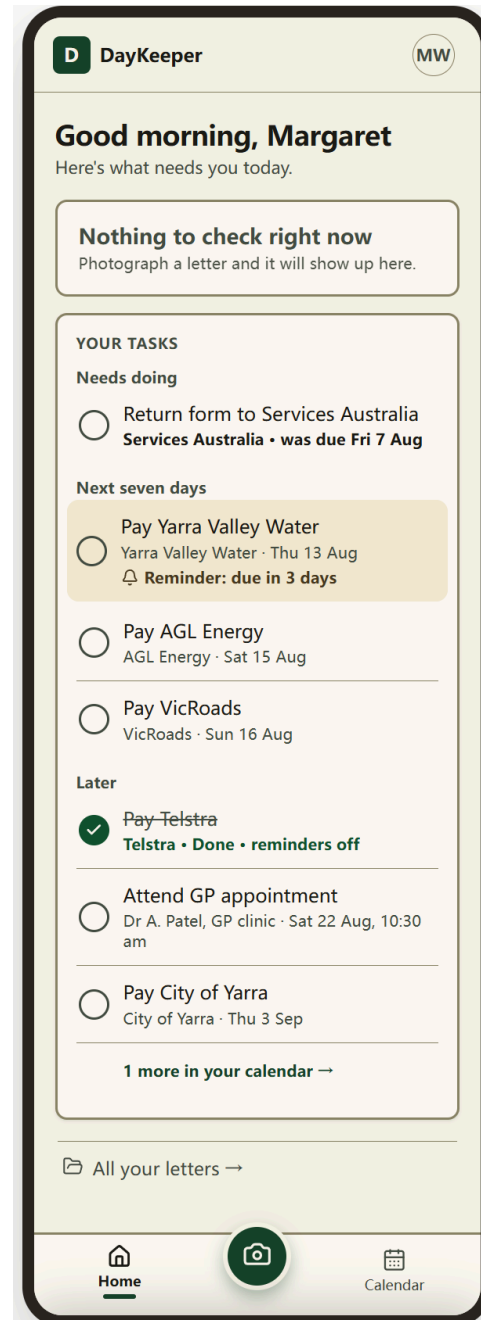
DayKeeper

A walk through the product, and the machinery behind each screen

This document follows one letter from the moment it is photographed to the moment the task it became is ticked off. Every screen you see is the working prototype; under each one is what the system does, which tables it writes, and why it was designed that way rather than another way.

It describes the shape of the system, not the state of the work. The board says who is doing what this week; this says what the thing is.

Written by Junchun Zhang, technical lead
For Dr Golnoush Abaei (supervisor), and the team
Screens the clickable prototype, docs/prototype/user/
Version 1.0



The letter by the front door

Margaret Wilson is 78. She lives alone in a brick house in the northern suburbs of Melbourne, her cataracts are getting worse, and every bill she has still arrives on paper. Her daughter is in Sydney and helps by phone. She has a smartphone, and she uses it for calls and for photographs.

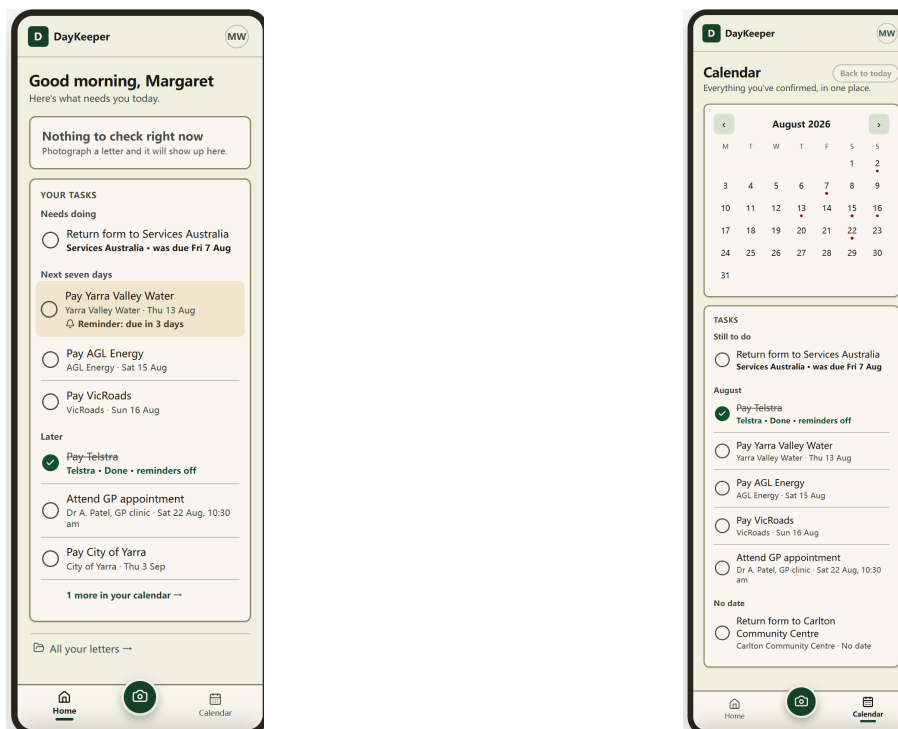
A letter arrives. It is a bill, or a form, or an appointment. Somewhere in it is a date, and somewhere else is the one thing she has to do before that date. The letter is designed to be read by a person with time and good light, and it will sit by the front door until she has both.

What goes wrong is not that she cannot read it. It is that reading it is not enough. The date has to survive the week between reading the letter and acting on it, and nothing in the house is doing that job. Her daughter's phone calls are doing that job.

This product does one thing: it makes the date outlive the letter. She photographs the letter, and the date it carries becomes a task, a place on a calendar, and three reminders. That is the whole of it.

One thing to know before the screens. The screenshots were taken from an earlier version of the prototype, which kept a fixed fictional today: **Monday 10 August 2026**. Every date in them is relative to that. The water bill was due on the fifth, so it is late. The pension form is due on the eleventh, so it is not. The prototype now works from the day it is opened, and ?today= in its address pins another. Its toolbar says which day it is on, so nobody has to guess.

What she gets back



The list, and the calendar drawn from it. Overdue is told in words at the top rather than in red, because colour is never the only signal here.

One list of what to do and the day it has to be done, and a calendar drawn from that same list. From where she is standing, that is the whole product.

Two verbs run the whole thing: photograph, and tick. She never types a date. She never sorts pages, files anything, names a document, or opens a settings screen. Everything else in this document exists to keep that true.

Nothing happens until you say so, and never from a date you haven't checked.

That promise is printed on the review screen. It does not hold because anyone is careful. It holds because the system has no path around her, and the next part walks every step there is.

Who she is matters to the design more than usual, because the people this product is for are the ones least able to absorb a mistake it makes. Eyes over seventy, a memory that is not reliable, and a lifetime of assuming that when a computer says no it is their own fault. So body text clears 7:1 contrast rather than the usual 4.5:1, nothing is blue, colour is never the only signal, and an error message never opens by describing something she did.

What this release is, and what it is not

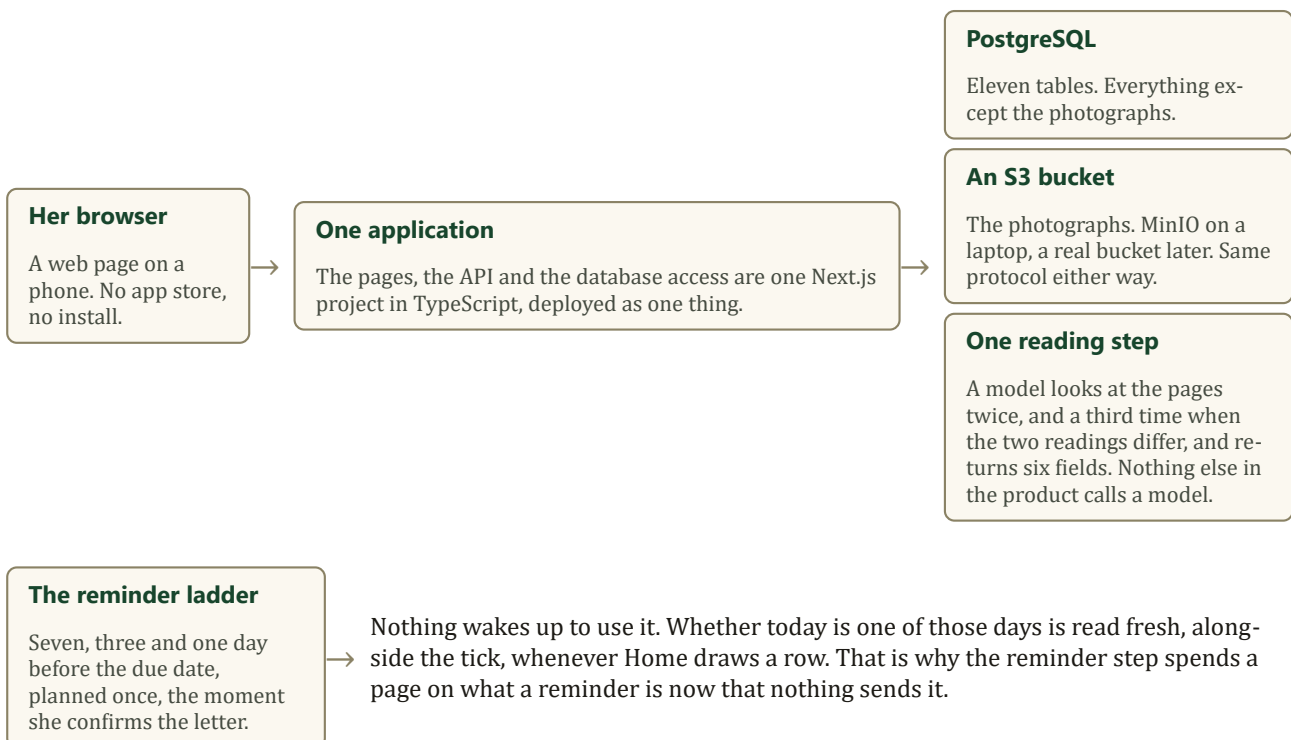
This is the first release, and it is deliberately narrow. **One upload is one letter**, which may run to several pages.

A larger design was drawn first. It took a pile of mixed post at once, worked out where one letter ended and the next began, and joined a later upload to a letter already in the system. Each of those needed its own model call. Each could be wrong without anyone noticing. Together they were most of the engineering. That design is the one a funded team builds. This is the one five students finish in a semester, and the rest is written down and deferred rather than forgotten.

What is left is one reading step whose answer has a fixed shape, and after that a system made of tables, dates and a checkbox. The full list is `docs/scope.md`, which every other document in this repository defers to.

The pieces

Six things do all the work. Everything later in this document is one of them doing its job.



One application, one language. The pages, the route handlers and the database access are one Next.js project in TypeScript.

The gain is in `src/lib/contract/`. The browser and the server import the same types. Change the shape of a response and both sides stop compiling. Anything the browser must never see is marked `server-only`, so a

leak is a build error instead of a surprise. One dependency set, one deployment, one test run, and one place to look when something breaks.

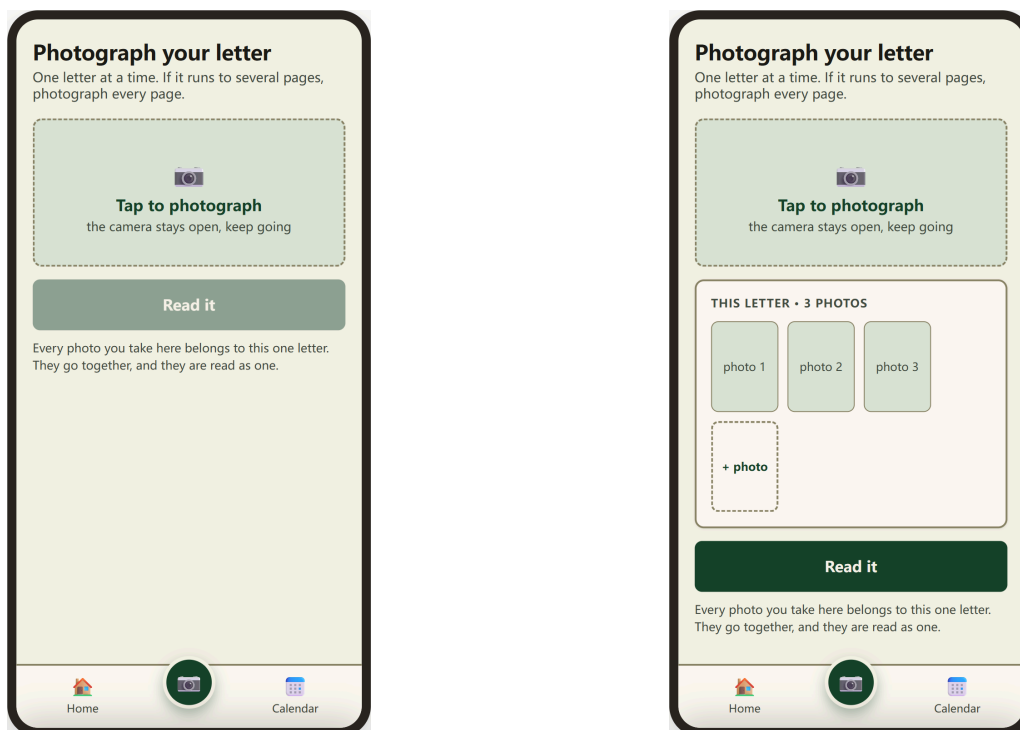
One letter, from the front door to the tick

The quickest way to understand DayKeeper is to watch it handle one letter. This part does that. The letter is an electricity bill from AGL: three pages, \$347.60, due Saturday 15 August. Today, in every screenshot, is Monday 10 August.

Each step names the tables it writes. One step writes nothing at all, and it is the most important screen in the product.

STEP 1 She photographs the letter

Writes: documents document_pages



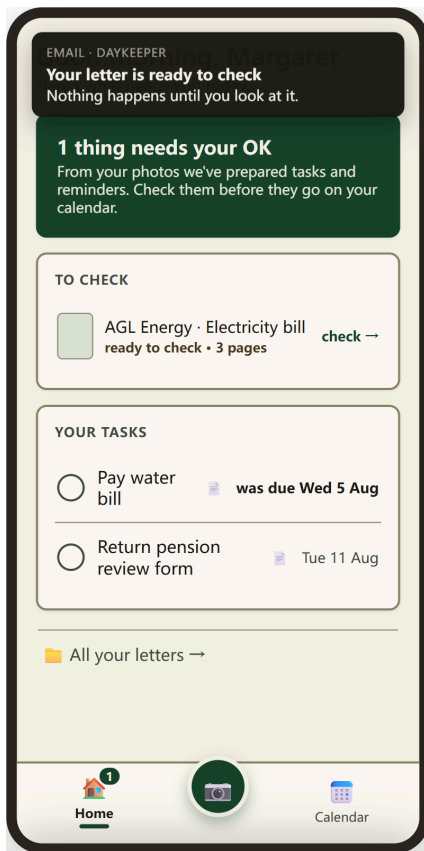
The camera stays open across taps, so a three page letter is three taps rather than three trips. The upload button stays dim until there is at least one page.

She taps the camera, photographs the three pages, and sends them. That is her whole job here. She never says what the letter is, and she never will.

Two kinds of rows are written. One row in `documents`: the letter exists. Three rows in `document_pages`: one per photograph. The photographs themselves go to the bucket, not into the database. A photograph is large and never changes, so the browser fetches it straight from the bucket, and the row keeps only the key for finding it.

STEP 2 The system reads it

Writes: `extraction_runs` `model_calls` `extracted_fields` `extracted_identifiers`



The upload screen closes at once. She puts the phone down.

About twenty seconds later it buzzes: *Your letter is ready to check. Nothing happens until you look at it.*

In those twenty seconds the product did its one clever thing. A vision model looked at the three photographs, twice, and answered six questions: what kind of letter this is, who sent it, what she has to do, by when, how much, and what reference to quote. For this bill: an electricity bill, from AGL Energy, pay the amount due, by 15 August, \$347.60, reference 9201 4471 88.

The six questions, and the rule that a reader must answer all six every time, are in `src/lib/contract/fields.ts`. Every call is a row in `model_calls`, with what it answered and what it cost, under its round in `extraction_runs`, so a wrong answer can always be traced to the call that produced it. The prompt is one file, `src/server/extraction/prompt.md`.

On this bill the model was sure of all six. It is not always. Each answer carries a verdict: confirmed, or uncertain when it has a value it is not sure of, or unreadable when it has nothing.

The reading finished while the phone was in her bag. The message is what brings her back.

Six fields, three states, and the one she never sees



One letter, photographed read by the model, and nobody is asked to fill anything in

→
the model answers with all six, every time

What storage keeps
extracted_fields, one row per field, six every time

document_type	Electricity bill	CONFIRMED
issuer	Example Energy	CONFIRMED
action_required	Pay the total amount due	CONFIRMED
due_date	24 Aug 2026	CONFIRMED
amount	\$82.42	CONFIRMED
reference	4471 88	UNCERTAIN

The hedged value is not thrown away
It stays in `extracted_fields`, where the accuracy work can count how often the model hedges and what it guessed when it did. That measurement is the only reader it has.

the server collapses uncertain to unreadable on the way out

What any screen can show
the API response, the review screen, and the calendar

document_type	Electricity bill	CONFIRMED
issuer	Example Energy	CONFIRMED
action_required	Pay the total amount due	CONFIRMED
due_date	24 Aug 2026	CONFIRMED
amount	\$82.42	CONFIRMED

the reference row is not drawn at all; an empty row invites an answer nobody is asking for

A date or an amount is never left out like this
Left out, the card would say the letter has no date or nothing to pay. If the model is not sure of either, the reading fails and she sees the red row instead.

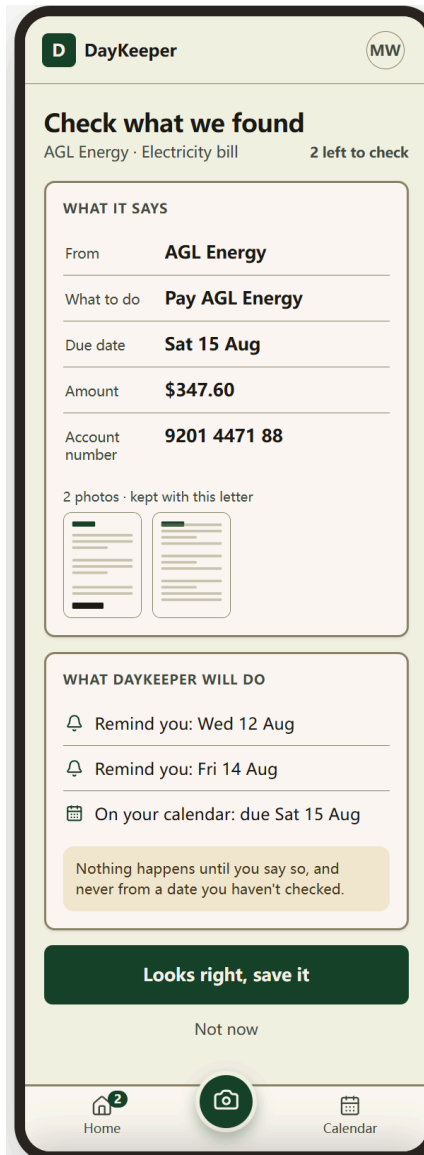
A second bill, written to go wrong: the model was not sure of the reference number.

A value the model was not sure of is never offered as a guess. Show her a half sure number in grey and she will approve it with a nod. So the row is not drawn at all. A date or an amount is different: left out, the card

would say the letter has no date or nothing to pay. So if the model is not sure of either, the reading fails and she sees the red row instead. A date nobody really checked has no way to reach the calendar.

STEP 3 She checks it

Writes: *nothing at all*



The whole screen. Nothing on it is editable, and nothing on it is a question.

She taps the letter and gets this screen. The top card is what the letter says. The bottom card is what DayKeeper will do about it: remind her on Wednesday the 12th and Friday the 14th, and put the due date on her calendar. One green button.

Her whole job is to hold the letter in one hand and check the screen against it. Recognition is something a tired person with poor eyesight can still do reliably. Typing is not, so there is nothing to type.

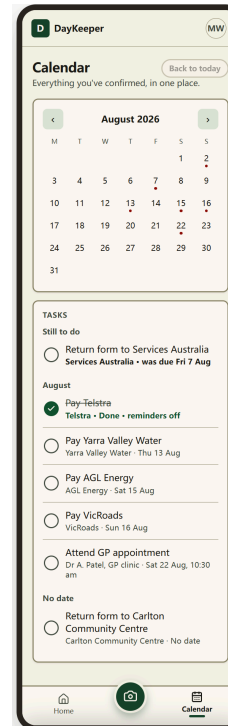
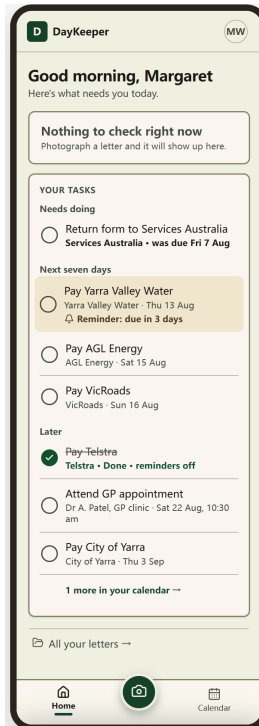
And this step writes nothing to the database. The screen shows, it never asks, so there is no answer to record.

The reminder days follow one rule: seven days before the due date, three days before, and one day before, whichever of those are still ahead of today. Seven days before this bill's due date was 8 August. That day has already gone by, so the card offers two days instead of three. The card and the later rows are planned by the same function, so the screen can never promise a day that never comes.

The sentence under the plan is the product's one promise. The next two steps are how it is kept.

STEP 4 Her yes writes the rows

Writes: tasks reminders documents



The same bill, now on the list, and its due date in red on the calendar. The gold on Pay Yarra Valley Water belongs to a different task: today happens to be three days before that one is due.

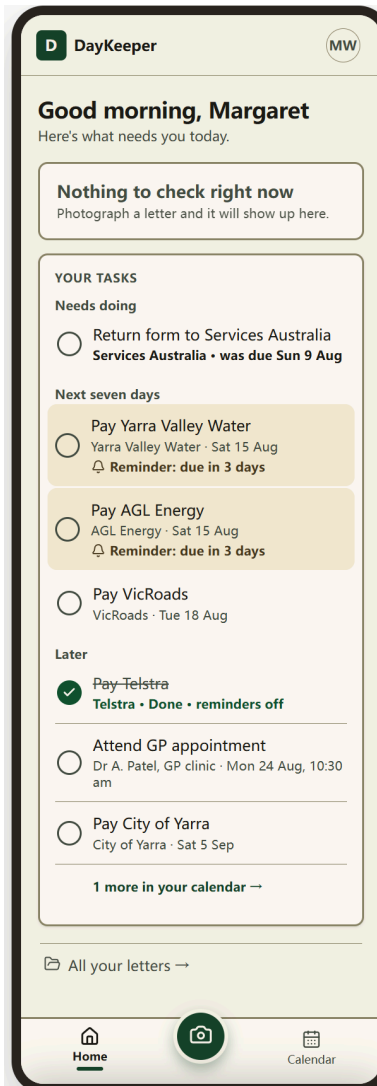
She taps **Looks right, save it**. Three writes happen together. One row in tasks: pay AGL electricity bill, due 15 August. Two rows in reminders: the 12th and the 14th, exactly the days the card named. And the letter's row in documents is marked confirmed. A short note says where the task went, and the screen moves on by itself: to the next letter waiting, or, after the last one, to the calendar at the month of the task she just saved. A letter that asks for nothing makes no task at all and is simply kept in her letters.

This tap is the only thing in the product that turns a reading into something that will act later. Before it, the bill was rows the model wrote and a screen she could look at. After it, there is a task on her list, and two rows in reminders waiting for their day to come.

Everything on the calendar got there the same way: a sure reading that a person looked at. There is no import, no guess, and no other path. That is how the promise on the review screen is kept. Not by anyone being careful, but because no other path exists.

STEP 5 A reminder day arrives

Writes: *nothing at all*



Wednesday 12 August, Home open for some other reason. The AGL row is the one she already knows, one line taller.

She opens the app on Wednesday, and the AGL row looks different: a pale gold tint, a bell, and *Reminder: due in 3 days*. Nothing buzzed and nothing arrived. The mark is there because today is one of the three days reminders named for this task, and for no other reason.

Drawing it costs one read, not a write: is today among this task's reminder days, and is the task still open. Both answers already exist in rows written at confirm, so the step that puts the mark on screen writes nothing at all, the same as checking a letter does.

Had she not opened the app that day, nothing would have happened, and nothing tries again later. There is no push, no email, and nothing running while the phone is asleep. Opening Home is the only way she ever sees a reminder, on the day it falls.

Whether a row is marked is read fresh, never stored

Home, opened
on each of the task's
three reminder days

11 Sep
a reminder day for this task

reminders has today, task is
open
MARKED the row is tinted, on
this draw

15 Sep
a reminder day for this task

reminders has today, task is
completed
**NOT
MARKED** the row draws like
any other done row

17 Sep
a reminder day for this task

reminders has today, task is
open
MARKED the row is tinted
again, on this draw

September 2026
tasks.state, painted along
the days



Margaret
ticking and unticking

12 Sep
she ticks it off
tasks.state = completed

16 Sep
she changes her mind and
unticks it
tasks.state = open

Every visit reads the same two facts and nothing else: does **reminders** have a row for today, and is **tasks.state** still open. Neither the 12th's tick nor the 16th's untick touches the reminders table, so there is nothing to get out of step.

reminders				written once, at confirm, and never again
id	task_id	remind_on	created_at	
...a1	t-9201	2026-09-11	4 Sep, at confirm	
...a2	t-9201	2026-09-15	4 Sep, at confirm	
...a3	t-9201	2026-09-17	4 Sep, at confirm	

Three rows, planned when she confirmed the letter and unchanged since. Whether the 11th, the 15th or the 17th put a mark on her screen was decided each time Home drew, by reading this table alongside the tick, not by writing to it.

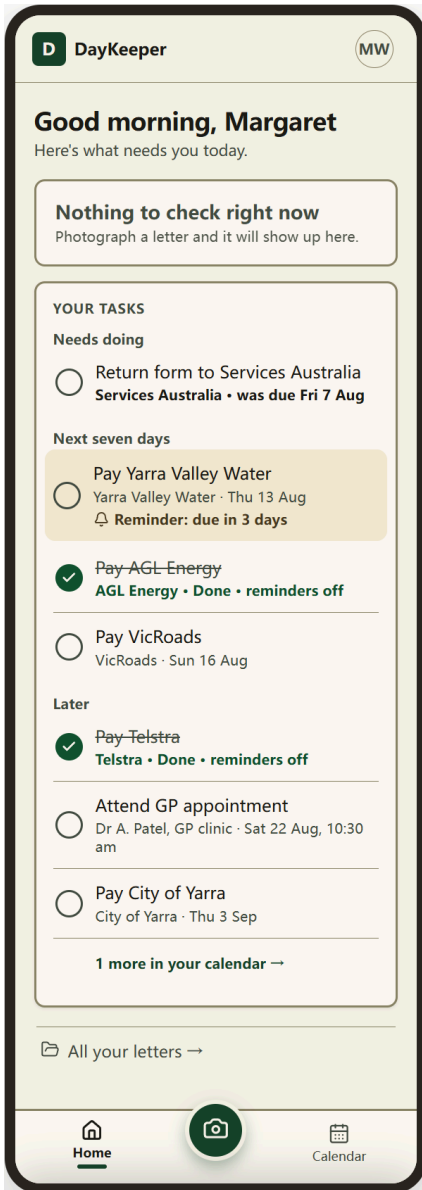
A different bill through a fickle week: ticked, unticked, and read fresh each time Home happens to be opened on one of its three reminder days.

Why marking is read fresh, instead of the tick cancelling reminders. She ticks the bill at breakfast, unticks it at lunch, ticks it again at dinner. What must the system do to Friday's mark each time? Nothing. Whichever way the tick sits on Friday is what Friday's draw will show.

The design this replaced did the opposite: a reminder was a message queued to go out, so ticking had to cancel the ones still waiting, and unticking had to revive them, except the ones whose moment had already passed. The same fact kept in two places, and rules to keep the copies agreeing. Reading the tick at the moment it is drawn keeps it in one place, and lets her change her mind for free.

STEP 6 She ticks it

Writes: tasks



Struck through, still on the list, and told in words: done, reminders off.

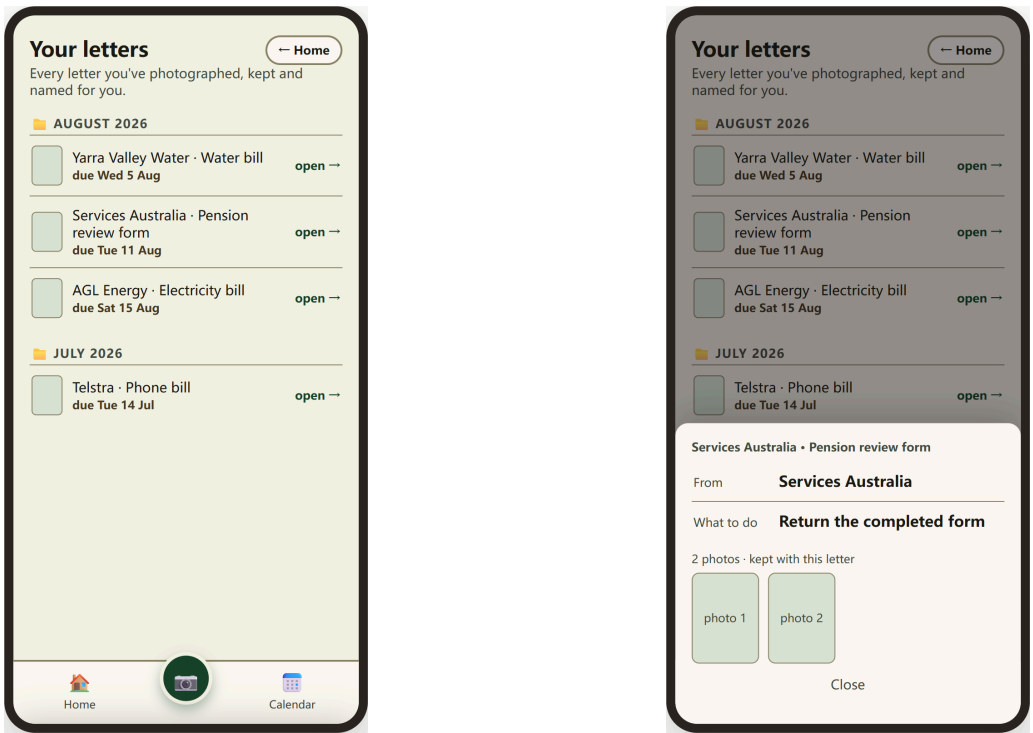
She pays the bill on Wednesday and ticks it off. One column of one row changes: the tick on the task.

On Friday, its last reminder day, Home reads the task, finds the tick, and draws the row plainly, the same as any other done task. Had she left it unticked, Friday would have looked exactly like Wednesday did. That is all “reminders off” is. It is not a switch stored anywhere. It is true because the tick is set, and it stops being true the moment she unticks.

The ticked task stays on the list for a week, struck through, because a task that vanishes looks exactly like a task that was lost. Its calendar entry stays forever, so “did I pay that” still has an answer in December.

Overdue works like reminders off: worked out, never stored. The pension review form at the top of her list says *was due Fri 7 Aug* because today is the 10th and nobody has returned it. Every screen makes that comparison in Melbourne time at the moment it draws, so nothing goes overdue at eleven at night because a server sits in another country.

The letters she has kept



Every letter that has been read, and one of them opened: what was read, and the photographs it was read from.

Nothing in this area asks anything of her. It exists because a letter is evidence. The paper itself may well be in the bin, so the photographs are the only copy. Opening a letter here is also the only way to check what the system read against the original.

When the reading fails

About one letter in eight cannot be read at all. During development the model is played by a stand-in reader, and the stand-in fails on purpose at roughly that rate, so the failure path is exercised every day.

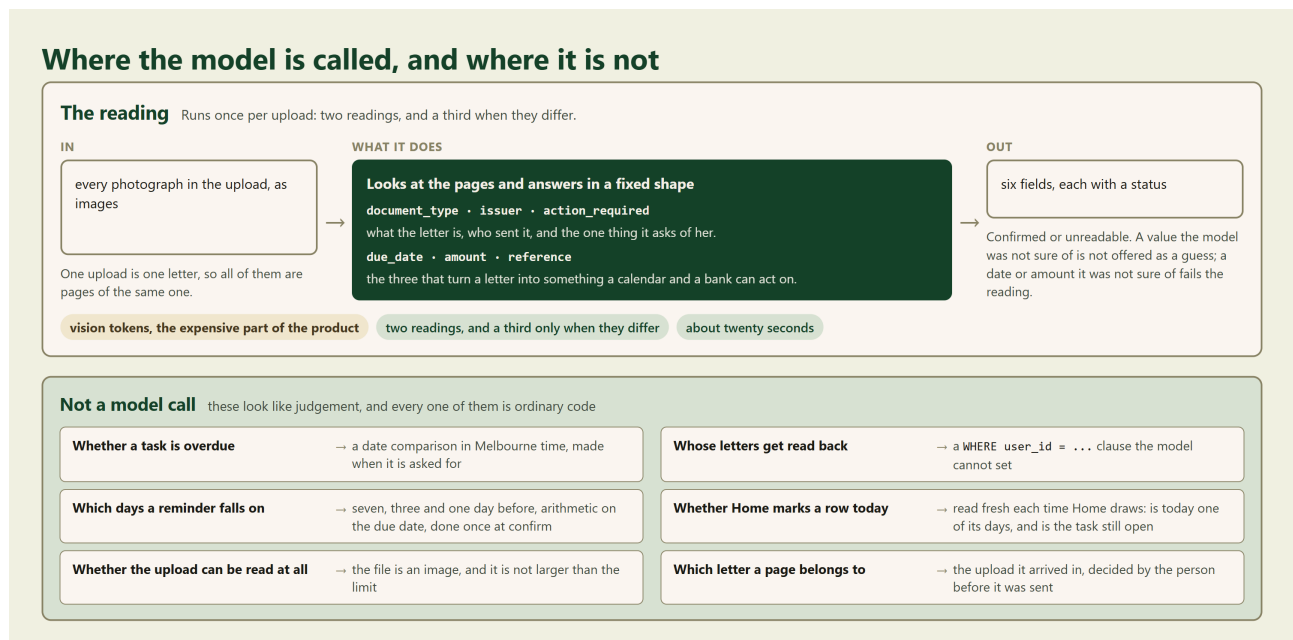
A model call that goes wrong is made again first, up to three tries in all, while the letter still says “reading...”. Each try is its own row in `model_calls`, so the table shows how many calls a letter took. When the two readings and the judge all disagree, the whole letter is read again, up to five rounds, each its own row in `extraction_runs`. Only when the last try or the last round fails is she told.

After that, a failed reading is the end of the road. The letter keeps its photographs and says plainly that it could not be read. There is no prompt to photograph it again, and nothing that suggests the failure was her fault.

A smaller promise, made honestly. An earlier design offered her a way out of every failure: photograph it again, or a screen explaining what went wrong. Every one of those paths needed the system to know why it had failed, and a model that cannot read a page usually cannot say why. A remedy that does not work is worse than none, so this release offers none and says so.

Where the model is called

One place in the whole product calls a model: the reading step, which reads the letter twice and a third time when the two readings differ. Everything after it is ordinary code and ordinary SQL.



The one reading step, and six things that look like judgement and are not.

Why the model is given the photographs. A letter is drawn for a human eye, and half of what it means is in the arrangement rather than the words. The number that matters sits in a box. The heading is bold. Five ways to pay sit side by side, each with its own reference number, and the only thing saying which number goes with which is the column it is in. A model that looks at the page sees all of that.

The bottom half of that diagram is the more useful one. Six questions that sound like they need judgement. Every one of them is a line of ordinary code.

The eleven tables

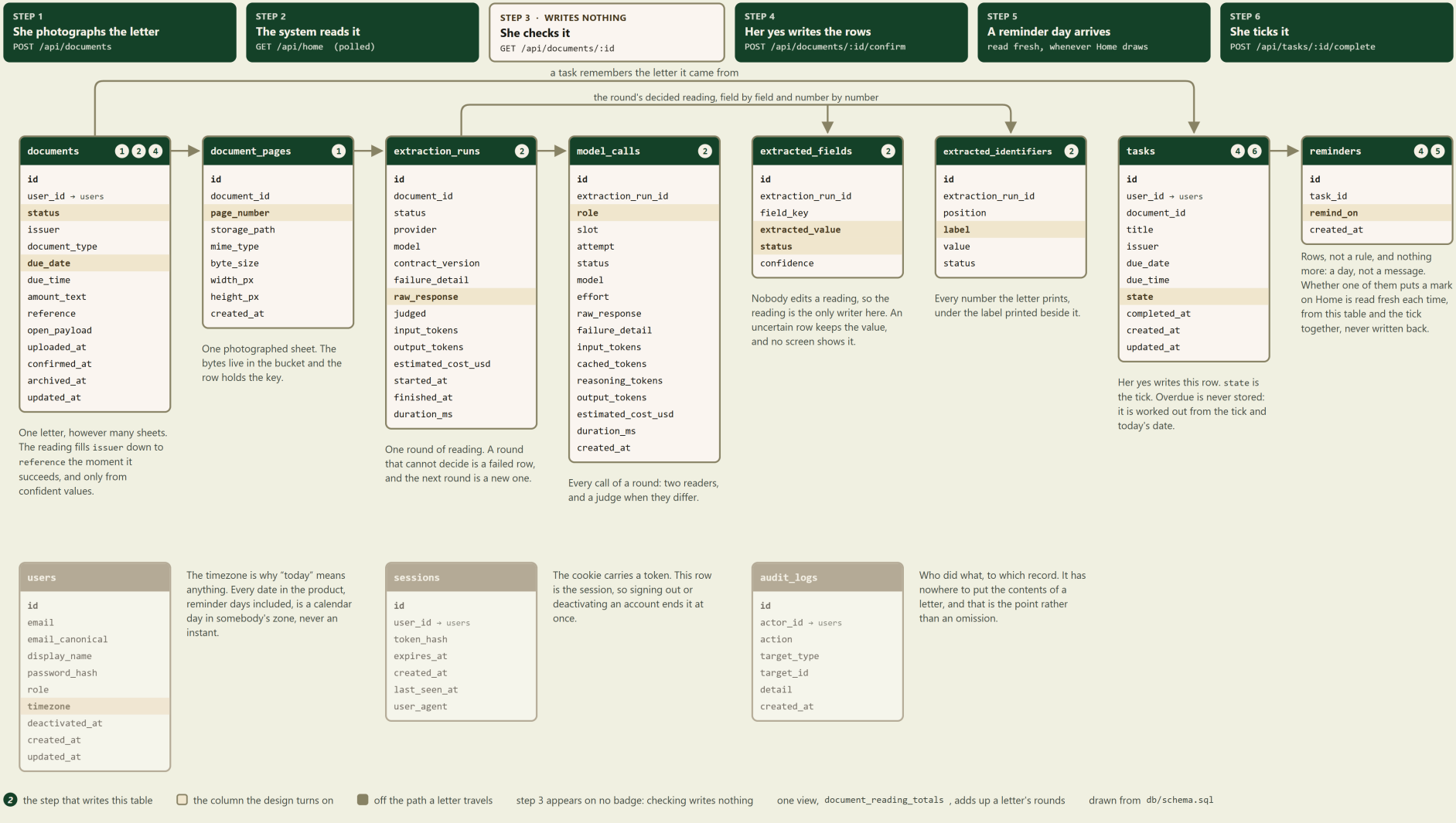
The next page is the whole database at once, laid out along the same journey as the previous part. Two absences are worth knowing before you look, because an absence is easy to read past.

Step three is on no badge. The step where she checks the reading writes nothing. The screen shows, it never asks, so there is no answer to record. A design that asks questions needs somewhere to store the answers. This one asks none.

Nothing derivable is stored. There is no overdue column, no reminders-off flag, and no count of anything. Each would be a second copy of a truth that already exists, and a second copy can disagree with the first. Overdue is a comparison made when somebody asks. Reminders off is the tick, read at the moment Home draws the row.

The definition of every column, and the reason it exists, is in `db/schema.sql`. That file is not a description of the database. It is the database: there are no migrations, so editing it and running `npm run db:reset` is the whole of changing the schema.

The eleven tables, and which step writes each



Eleven tables, every column of each. The path a letter travels runs left to right along the top row, and the grey row underneath never touches a letter.

What is not settled

This document describes a design, and a design has edges. These are the ones that are known rather than the ones that are hidden.

Open question	What is known about it
Sending any email at all	Nothing in the product sends one today, and a reminder never will (KAN-62): it is a mark on Home, not a message. Password reset still needs a sender, and one is not chosen yet.
The upload at its limits	A page count and a size are checked. What the screen does when a photograph is enormous, or when the connection drops halfway, is not drawn.
Rate limiting on sign in	The password hashing is deliberate and slow. The number of attempts is not yet bounded.
Which model	The reading is written against an interface rather than a provider, so the choice is a swap of one file. Which model, and how it is paid for, is being settled outside this document.

Where the answers live

Every fact here has one home, and the home is usually the thing itself. This table is how to find it. If a document and the code disagree, the code is right. That is why the reasoning was put in the code.

If you want to know	Open
what this release builds, and what it does not	<code>docs/scope.md</code>
how to run the whole thing	<code>docs/start-here.md</code>
what each endpoint takes and returns	<code>docs/api.md</code>
the six fields, and why six is a floor	<code>src/lib/contract/fields.ts</code>
what a field status means, and <code>open_payload</code>	<code>src/lib/contract/extraction.ts</code>
why the review screen has no inputs	<code>src/lib/contract/api.ts</code>
the 7/3/1 reminder rule, and what a reminder actually is	<code>src/lib/contract/reminders.ts</code>
why every date is Melbourne and date only	<code>src/lib/contract/dates.ts</code>
every table and every column, with reasons	<code>db/schema.sql</code>
what a reader has to promise	<code>src/server/extraction/provider.ts</code>
how the mock reader misbehaves, and why	<code>src/server/extraction/mock-provider.ts</code>
sessions, and why the passwords are hashed that way	<code>src/server/auth/</code>
where the photographs live	<code>src/server/storage.ts</code>
the palette, the contrast, the type sizes	<code>docs/theme.md</code>
the conventions, and why this is one application	<code>AGENTS.md</code>

About to change something? Read `docs/scope.md` first to see whether it is in this release at all, then find the file above. This document is a tour, and a tour is not a source: it is rebuilt from those files, never the other way round.